Projeto e análise

Discente: Bruno Victor Paiva da Silva

4. **Mostre numericamente com suas implementações dos algoritmos de multiplicação de matrizes que o algoritmo de Strassen é mais rápido que o algoritmo convencional.**

```
#include <iostream>
#include <vector>
#include <chrono>
#include <fstream>

using namespace std;

using Matrix = vector<vector<int>>;

Matrix add(const Matrix& A, const Matrix& B) {
    int n = A.size();
    Matrix C(n, vector<int>(n));
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            C[i][j] = A[i][j] + B[i][j];
    return C;
}

Matrix subtract(const Matrix& A, const Matrix& B) {
    int n = A.size();
    Matrix C(n, vector<int>(n));
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            C[i][j] = A[i][j] - B[i][j];
    return C;
}
```

```
Matrix conventional_multiply(const Matrix& A, const Matrix&
B) {
    int n = A.size();
    Matrix C(n, vector<int>(n, 0));
        for (int i = 0; i < n; i++)
            for (int k = 0; k < n; k++)
                for (int j = 0; j < n; j++)
                    C[i][j] += A[i][k] * B[k][j];
        return C;
}

Matrix strassen(const Matrix& A, const Matrix& B) {
    int n = A.size();
    if (n <= 64) {
        return conventional_multiply(A, B);
    }
    int newSize = n / 2;
    Matrix A11(newSize, vector<int>(newSize));
    Matrix A12(newSize, vector<int>(newSize));
    Matrix A21(newSize, vector<int>(newSize));
    Matrix A22(newSize, vector<int>(newSize));
    Matrix B11(newSize, vector<int>(newSize));
    Matrix B12(newSize, vector<int>(newSize));
    Matrix B21(newSize, vector<int>(newSize));
    Matrix B22(newSize, vector<int>(newSize));

  for (int i = 0; i < newSize; i++) {
      for (int j = 0; j < newSize; j++) {
          A11[i][j] = A[i][j];
          A12[i][j] = A[i][j + newSize];
          A21[i][j] = A[i + newSize][j];
          A22[i][j] = A[i + newSize][j + newSize];
          B11[i][j] = B[i][j];
          B12[i][j] = B[i][j + newSize];
          B21[i][j] = B[i + newSize][j];
```

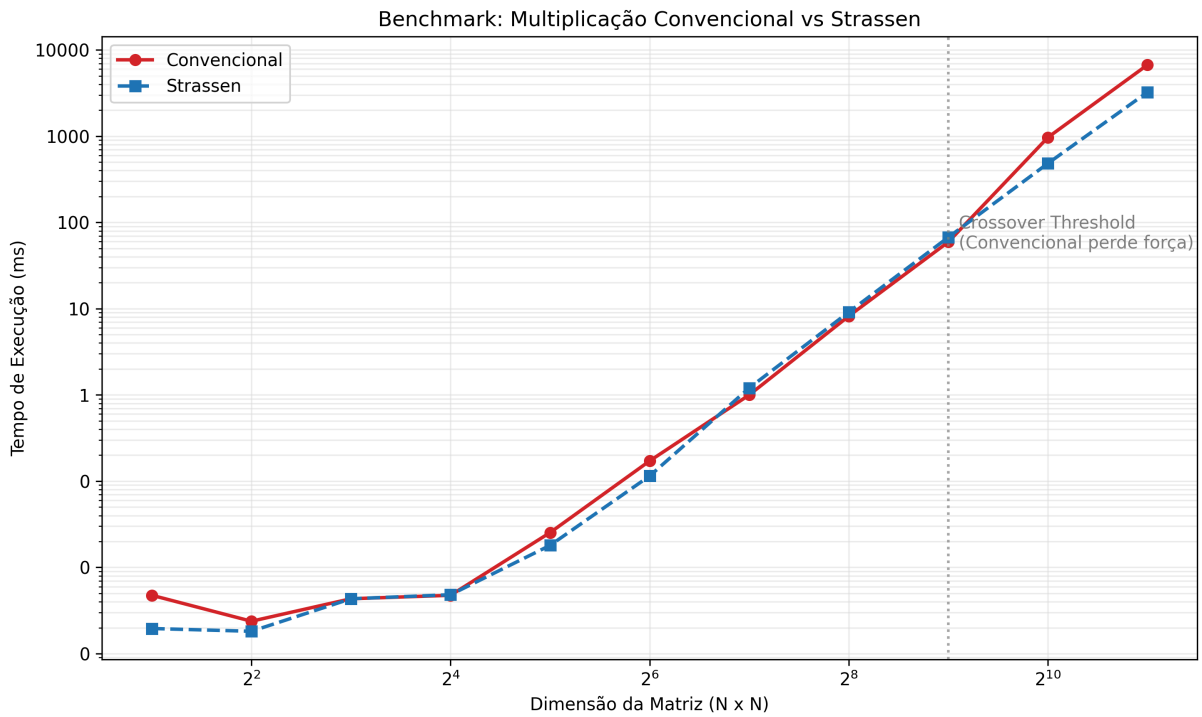
```
            B22[i][j] = B[i + newSize][j + newSize];
        }
    }
  Matrix M1 = strassen(add(A11, A22), add(B11, B22));
  Matrix M2 = strassen(add(A21, A22), B11);
  Matrix M3 = strassen(A11, subtract(B12, B22));
  Matrix M4 = strassen(A22, subtract(B21, B11));
  Matrix M5 = strassen(add(A11, A12), B22);
  Matrix M6 = strassen(subtract(A21, A11), add(B11, B12));
  Matrix M7 = strassen(subtract(A12, A22), add(B21, B22));

  Matrix C(n, vector<int>(n));
  for (int i = 0; i < newSize; i++) {
      for (int j = 0; j < newSize; j++) {
          C[i][j] = M1[i][j] + M4[i][j] - M5[i][j] + M7[i]
[j];
        }
    }
    for (int i = 0; i < newSize; i++) {
        for (int j = 0; j < newSize; j++) {
            C[i][j + newSize] = M3[i][j] + M5[i][j];
        }
    }
    for (int i = 0; i < newSize; i++) {
        for (int j = 0; j < newSize; j++) {
            C[i + newSize][j] = M2[i][j] + M4[i][j];
        }
    }
    for (int i = 0; i < newSize; i++) {
        for (int j = 0; j < newSize; j++) {
            C[i + newSize][j + newSize] = M1[i][j] - M2[i][j]
+ M3[i][j] + M6[i][j];
        }
    }
    return C;
}
```

```
template<typename Func>
double run_benchmark(Func func, const Matrix& A, const Matrix
& B) {
    auto start = chrono::high_resolution_clock::now();
    func(A, B);
    auto end = chrono::high_resolution_clock::now();
    return chrono::duration<double, milli>(end - start).count
();}
int main() {
    ofstream output("strassen_benchmark.csv");
    output << "Size,Conventional,Strassen\n";
    for (int n = 2; n <= 2048; n *= 2) {
        Matrix A(n, vector<int>(n)), B(n, vector<int>(n));
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++) {
                A[i][j] = rand() % 10;
                B[i][j] = rand() % 10;
            }
        double conventional_time = run_benchmark(conventional
_multiply, A, B);
        double strassen_time = run_benchmark(strassen, A, B);
        output << n << "," << conventional_time << "," << str
assen_time << "\n";
   }
   output.close();
   return 0;
}
```

Size	Convencional	Strassen
2	0,004749	0,001956
4	0,002375	0,001816
8	0,004331	0,004331
16	0,004749	0,00482

32	0,025352	0,01802
64	0,171743	0,114961
128	1,00057	1,20548
256	8,20965	9,13996
512	59,422	67,3548
1024	969,342	484,441
2048	6722,12	3243,8



5. **Escolha um algoritmo recorrente para aplicar um dos 4 métodos de resolução de**
   recorrência descritos no capítulo 4 para medir o custo da recorrência do algoritmo escolhido. Compare o resultado com medições de tempo.

```
#include <iostream>
#include <vector>
#include <chrono>
#include <fstream>
```

```
#include <numeric>
#include <random>

using namespace std;
using KeyType = int;

KeyType binary_search_recursive(const vector<KeyType>& v, Key
Type key, int begin,

int end) {
    if (begin > end) return -1;

    int middle = begin + (end - begin) / 2;

    if (key < v[middle]) {
        return binary_search_recursive(v, key, begin, middle
- 1);
    } else if (key > v[middle]) {
        return binary_search_recursive(v, key, middle + 1, en
d);
    } else {
        return middle;
    }
}

int main() {
    ofstream csv("benchmark_busca.csv");
    csv << "n,tempo_ns\n";

    unsigned int max_n = 1 << 28;
    int repeticoes = 50000;
    mt19937 gen(42);

    for (unsigned int n = 16; n <= max_n; n *= 2) {
        vector<KeyType> v(n);
        iota(v.begin(), v.end(), 1);
```

```
        uniform_int_distribution<KeyType> distrib(1, n);
        vector<KeyType> chaves(repeticoes);

        for(int i = 0; i < repeticoes; i++) {
            chaves[i] = distrib(gen);
        }

        volatile KeyType sink = 0;
        auto start = chrono::high_resolution_clock::now();

        for (int i = 0; i < repeticoes; i++) {
            sink = binary_search_recursive(v, chaves[i], 0, n
- 1);
        }

        auto end = chrono::high_resolution_clock::now();
        chrono::duration<double, std::nano> elapsed = end - s
tart;
        double tempo_medio_ns = elapsed.count() / repeticoes;

        csv << n << "," << tempo_medio_ns << "\n";
    }

    csv.close();
    return 0;
}
```

Equação de recorrência:

$$T(n) = T\left(\frac{n}{2}\right) + \Theta(1)$$

Parâmetros do Teorema Mestre:

$$a = 1; b = 2; f(n) = \Theta(1)$$

Cálculo:

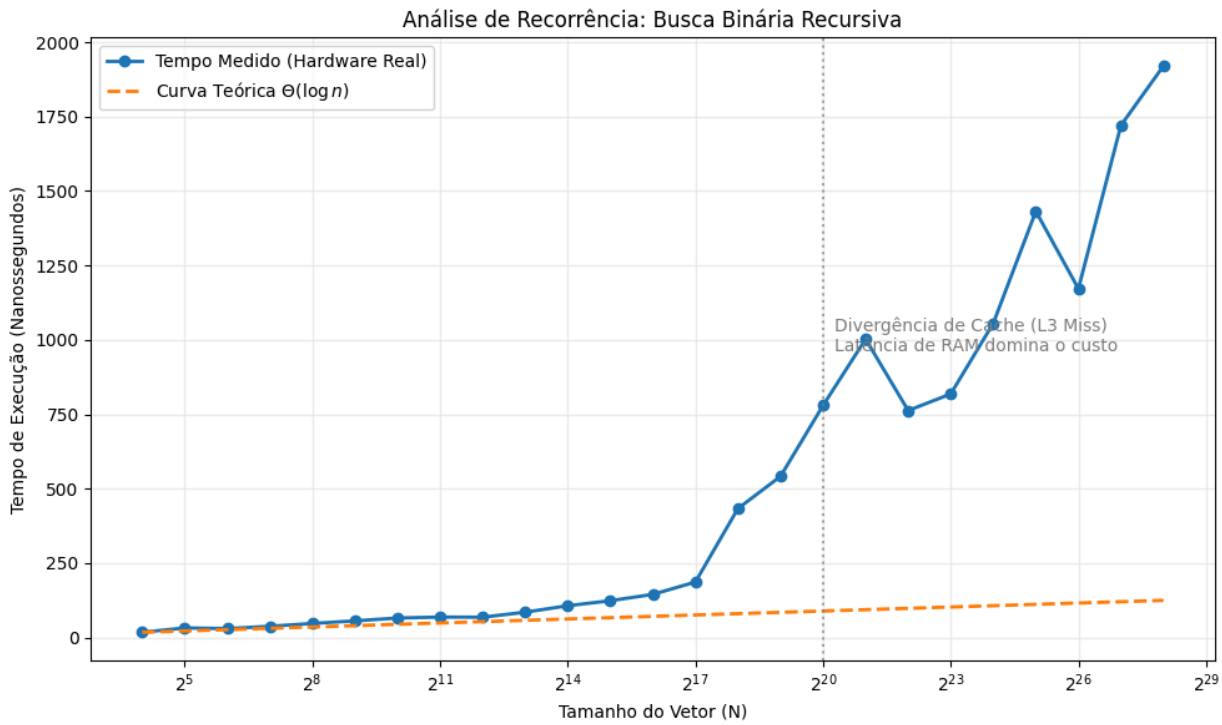
$$n^{\log_b a} = n^{\log_2 1} = n^0 = 1 = \Theta(1)$$

Conclusão:
Como

$$f(n) = \Theta(n^{log_b a})$$

, aplicamos o Caso 2 do Teorema Mestre:

$$T(n) = \Theta(n^{\log_b a} \cdot \log n) = \Theta(\log n)$$



6. **O problema de balanceamento de cargas busca atribuir tarefas de tamanhos**
   diferentes a trabalhadores, de modo a minimizar a carga máxima que um trabalhador irá executar. Em um problema em que temos n tarefas e k trabalhadores (n > k), considere que o balanceador irá distribuir as n/k primeiras
   tarefas para o primeiro trabalhador, as n/k tarefas seguintes para o segundo

trabalhador, e assim por diante. Mostre numericamente como permutar aleatoriamente os dados de entrada, que são as cargas de cada tarefa, pode influenciar na solução desse balanceador.

```
#include <iostream>
#include <vector>
#include <numeric>
#include <random>
#include <algorithm>
#include <iomanip>

using namespace std;

void distribute_and_print(const vector<int>& tasks, int worke
rs,
                                                        const str
ing& scenario_name) {
    int n = tasks.size();
    int chunksize = n / workers;
    vector<int> workload(workers, 0);
    int makespan = 0;

    for (int w = 0; w < workers; w++) {
        for (int i = w * chunksize; i < (w + 1) * chunksize;
i++) {
            workload[w] += tasks[i];
        }
        makespan = max(makespan, workload[w]);
    }

    cout << scenario_name << "\n";
    cout << setw(8) << "Worker" << " | " << setw(8) << "Workl
oad\n";
    for (int w = 0; w < workers; w++) {
        cout << setw(8) << w << " | " << setw(8) << workload
```

```
[w] << "\n";
    }
    cout << "-> Gargalo: " << makespan << "\n\n";
}

int main() {
    int n = 64;
    int workers = 4;
    vector<int> tasks(n);

    iota(tasks.begin(), tasks.end(), 1);
    int total_work = accumulate(tasks.begin(), tasks.end(),
0);

    cout << "Trabalho Total: " << total_work << "\n";
    cout << "Carga Ideal: " << total_work / workers << "\n
\n";

    distribute_and_print(tasks, workers, "Sequencial");

    random_device rd;
    mt19937 g(rd());

    for (int i = 1; i <= 4; i++) {
        shuffle(tasks.begin(), tasks.end(), g);
        distribute_and_print(tasks, workers, "Aleatoria " + t
o_string(i));
    }

    return 0;
}
```

```
bruno@pop-os:~/programacao-paralela/
Trabalho Total: 2080
Carga Ideal: 520

Sequencial
  Worker | Workload
       0 |      136
       1 |      392
       2 |      648
       3 |      904
-> Gargalo: 904

Aleatoria 1
  Worker | Workload
       0 |      469
       1 |      527
       2 |      512
       3 |      572
-> Gargalo: 572

Aleatoria 2
  Worker | Workload
       0 |      503
       1 |      665
       2 |      511
       3 |      401
-> Gargalo: 665

Aleatoria 3
  Worker | Workload
       0 |      508
       1 |      504
       2 |      648
       3 |      420
-> Gargalo: 648

Aleatoria 4
  Worker | Workload
       0 |      631
       1 |      539
       2 |      427
       3 |      483
-> Gargalo: 631
```